NumPy Master Notes (Beginner → Advanced)

1. Overview

NumPy (Numerical Python) is the most important Python library for:

- Numerical computing
- Scientific computing
- Data analysis
- Machine learning
- Deep learning
- Matrix operations
- Image processing

NumPy provides a powerful object called:

```
ndarray
```

which is:

- Faster than Python lists
- Memory efficient
- Supports vectorized operations
- Supports multi-dimensional arrays

---

2. Setup & Installation

Install NumPy

```
pip install numpy
```

Import NumPy

```
import numpy as np
```

Why use `np`?

`np` is just a short alias.

Instead of writing:

```
numpy.array()
```

we write:

```
np.array()
```

This is the standard convention used worldwide.

---

3. What is NumPy?

NumPy is a Python library used for:

- Arrays
- Matrix operations
- Mathematical calculations
- Statistics
- Linear algebra
- Data manipulation

The core object is:

```
ndarray
```

which stands for:

```
n-dimensional array
```

---

4. Why Use NumPy?

Problem with Python Lists

Python lists are:

- Slower
- Store mixed datatypes
- Require loops for operations

- Consume more memory

Example:

```
python_list = [1, 2, 3, 4, 5]

new_list = [x + 10 for x in python_list]
print(new_list)
```

Output:

```
[11, 12, 13, 14, 15]
```

Notice:

We had to manually loop through every element.

NumPy Advantage: Vectorization

What is Vectorization?

Vectorization means:

```
Performing operations on entire arrays WITHOUT writing loops.
```

Example:

```
import numpy as np

numpy_array = np.array([1, 2, 3, 4, 5])

print(numpy_array + 10)
```

Output:

```
[11 12 13 14 15]
```

Why is NumPy Faster?

NumPy arrays:

- Are stored continuously in memory
- Use optimized C code internally
- Avoid Python loop overhead
- Use SIMD/vectorized instructions

Performance Comparison

```
import numpy as np
import time

size = 1000000

python_list = list(range(size))
numpy_array = np.arange(size)

# Python list timing
start = time.time()
result_python = [x + 1 for x in python_list]
time_python = time.time() - start

# NumPy timing
start = time.time()
result_numpy = numpy_array + 1
time_numpy = time.time() - start

print(f"Python list addition: {time_python:.6f} seconds")
print(f"NumPy array addition: {time_numpy:.6f} seconds")
print(f"NumPy is {time_python/time_numpy:.1f}x faster!")
```

Example Output:

```
Python list addition: 0.035624 seconds
NumPy array addition: 0.001001 seconds
NumPy is 35.6x faster!
```

---

5. Creating NumPy Arrays

There are many ways to create arrays.

Method 1: From Python List

```
import numpy as np

arr1 = np.array([1, 2, 3, 4, 5])
print(arr1)
```

Output:

```
[1 2 3 4 5]
```

Why `np.array()`?

This converts:

```
Python list → NumPy ndarray
```

Method 2: Using `np.arange()`

```
arr2 = np.arange(0, 10, 2)
print(arr2)
```

Output:

```
[0 2 4 6 8]
```

Why use `arange()`?

It works like Python `range()` but returns a NumPy array.

Syntax:

```
np.arange(start, stop, step)
```

Method 3: Using `np.linspace()`

```
arr3 = np.linspace(0, 1, 5)
print(arr3)
```

Output:

```
[0.   0.25 0.5  0.75 1.  ]
```

Why use `linspace()`?

It creates evenly spaced values.

Syntax:

```
np.linspace(start, stop, number_of_values)
```

Useful in:

- Graph plotting
- Mathematics
- Simulations

Method 4: Arrays of Zeros

```
zeros = np.zeros(5)
print(zeros)
```

Output:

```
[0. 0. 0. 0. 0.]
```

Useful for:

- Initialization
- Machine learning
- Placeholder arrays

Method 5: Arrays of Ones

```
ones = np.ones(5)
print(ones)
```

Output:

```
[1. 1. 1. 1. 1.]
```

Method 6: Filled Arrays

```
full = np.full(5, 7)
print(full)
```

Output:

```
[7 7 7 7 7]
```

Method 7: Identity Matrix

```
identity = np.eye(3)
print(identity)
```

Output:

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

Why is Identity Matrix Important?

Identity matrix behaves like:

```
Number 1 in matrix multiplication
```

Method 8: Random Arrays

```
random_arr = np.random.rand(5)
print(random_arr)
```

Example Output:

```
[0.25 0.80 0.34 0.62 0.02]
```

---

6. NumPy Data Types (`dtype`)

NumPy arrays are homogeneous.

That means:

```
All elements must have SAME datatype.
```

Example

```
arr_int = np.array([1, 2, 3])
arr_float = np.array([1.0, 2.0, 3.0])
arr_mixed = np.array([1, 2.5, 3])

print(arr_int.dtype)
print(arr_float.dtype)
print(arr_mixed.dtype)
```

Output:

```
int32
float64
float64
```

Why did mixed array become `float64`?

Because NumPy automatically promotes datatypes.

Rule:

```
Safer datatype wins.
```

Since float can store integers but integer cannot store decimals:

```
int + float → float
```

Specifying Datatypes

```
arr = np.array([1, 2, 3], dtype=np.float32)
print(arr.dtype)
```

Output:

```
float32
```

Converting Datatypes

```
arr2 = arr.astype(np.float64)
print(arr2.dtype)
```

Output:

```
float64
```

Memory Understanding

Type	Bytes
int32	4
float32	4
int64	8
float64	8

Smaller datatype:

- Uses less memory
- Faster sometimes

---

7. Multi-Dimensional Arrays

NumPy supports:

- 1D arrays
- 2D arrays
- 3D arrays
- nD arrays

2D Array

```
matrix_2d = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])

print(matrix_2d)
```

Output:

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

Understanding Shape

```
print(matrix_2d.shape)
```

Output:

```
(3, 3)
```

Meaning:

```
3 rows, 3 columns
```

Dimensions (`ndim`)

```
print(matrix_2d.ndim)
```

Output:

```
2
```

Meaning:

```
This array has 2 axes/dimensions.
```

3D Array

```
matrix_3d = np.array([
    [[1, 2], [3, 4]],
    [[5, 6], [7, 8]]
])

print(matrix_3d)
```

Think of 3D Arrays as:

```
Stack of 2D matrices
```

or

```
Pages → Rows → Columns
```

---

8. Array Attributes

```
arr = np.array([
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12]
])
```

Shape

```
arr.shape
```

Output:

```
(3, 4)
```

Meaning:

```
3 rows, 4 columns
```

Size

```
arr.size
```

Output:

```
12
```

Meaning:

```
Total elements
```

---

ndim

```
arr.ndim
```

Output:

```
2
```

Meaning:

```
Number of dimensions
```

---

dtype

```
arr.dtype
```

Shows datatype.

---

itemsize

```
arr.itemsize
```

Meaning:

```
Memory used by ONE element
```

---

nbytes

```
arr.nbytes
```

Meaning:

```
Total memory consumed by array
```

Formula:

```
nbytes = size × itemsize
```

Transpose (`T`)

```
arr.T
```

Rows become columns.

9. Indexing and Slicing

Indexing means accessing elements.

Slicing means extracting portions.

1D Indexing

```
arr = np.array([10, 20, 30, 40, 50])

print(arr[0])
print(arr[-1])
```

Output:

```
10
50
```

Slicing

```
print(arr[1:4])
```

Output:

```
[20 30 40]
```

Why stop index excluded?

Python slicing rule:

```
[start : stop)
```

Stop is NOT included.

This makes:

- Length calculation easier
- Memory handling efficient

Step Slicing

```
print(arr[::2])
```

Output:

```
[10 30 50]
```

2D Indexing

```
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])

print(matrix[1, 2])
```

Output:

```
6
```

Meaning:

```
row=1, column=2
```

First Row

```
matrix[0, :]
```

Second Column

```
matrix[:, 1]
```

Submatrix

```
matrix[0:2, 0:2]
```

Output:

```
[[1 2]
 [4 5]]
```

Boolean Indexing

One of NumPy's most powerful features.

```
arr = np.array([1,2,3,4,5,6,7,8,9,10])

print(arr[arr > 5])
print(arr[arr % 2 == 0])
```

Output:

```
[ 6  7  8  9 10]
[ 2  4  6  8 10]
```

Why is Boolean Indexing Powerful?

Because we can filter arrays WITHOUT loops.

Used heavily in:

- Data science
- Machine learning
- Data cleaning

---

10. Array Operations

NumPy supports element-wise operations.

Element-wise Operations

```
import numpy as np

a = np.array([1,2,3,4])
b = np.array([5,6,7,8])

print(a + b)
print(a - b)
print(a * b)
print(a / b)
```

Output:

```
[ 6  8 10 12]
[-4 -4 -4 -4]
[ 5 12 21 32]
[0.2 0.33333333 0.42857143 0.5]
```

Why `*` is NOT matrix multiplication?

In NumPy:

```
* → element-wise multiplication
```

Matrix multiplication uses:

```
np.dot()
```

or

```
@
```

Scalar Operations

```
print(a + 10)
print(a * 2)
print(a ** 3)
```

Output:

```
[11 12 13 14]
[2 4 6 8]
[ 1  8 27 64]
```

Broadcasting

When NumPy automatically applies scalar operations across arrays.

Example:

```
a + 10
```

means:

```
Every element + 10
```

without loops.

Comparison Operations

```
print(a > 2)
print(a == 3)
print(a <= 2)
```

Output:

```
[False False  True  True]
[False False  True False]
[ True  True False False]
```

---

11. Array Manipulation

Changing array shapes.

---

reshape()

```
arr = np.arange(12)

reshaped = arr.reshape(3,4)
print(reshaped)
```

Output:

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

Why reshape must preserve total elements?

Because reshape only changes VIEW.

No data is removed or added.

So:

```
3 × 4 = 12
```

must match original size.

---

flatten()

```
flattened = reshaped.flatten()
print(flattened)
```

Output:

```
[ 0  1  2  3  4  5  6  7  8  9 10 11]
```

---

reshape with `-1`

```
auto_reshape = arr.reshape(2, -1)
print(auto_reshape)
```

Output:

```
[[ 0  1  2  3  4  5]
 [ 6  7  8  9 10 11]]
```

Why use `-1`?

`-1` means:

```
NumPy calculate dimension automatically
```

Very useful when data size changes dynamically.

Transpose

```
matrix = np.array([[1,2,3],[4,5,6]])

print(matrix.T)
```

Output:

```
[[1 4]
 [2 5]
 [3 6]]
```

Concatenation

```
a = np.array([[1,2],[3,4]])
b = np.array([[5,6],[7,8]])

print(np.concatenate((a,b), axis=0))
```

---

12. Understanding Dimensions and Axis

This is one of the MOST IMPORTANT NumPy concepts.

Key Concepts

Concept	Meaning
ndim	Number of dimensions
shape	Size of each dimension
axis	Direction of operation

2D Array Understanding

```
arr_2d = np.array([
    [1,2,3],
    [4,5,6],
    [7,8,9]
])
```

Shape:

```
(3,3)
```

Meaning:

```
3 rows, 3 columns
```

---

axis=0 vs axis=1

This confuses beginners a lot.

Important rule:

```
Axis tells WHICH dimension gets collapsed.
```

---

axis=0

```
np.sum(arr_2d, axis=0)
```

Output:

```
[12 15 18]
```

Calculation:

```
1+4+7 = 12
2+5+8 = 15
3+6+9 = 18
```

Why does axis=0 sum vertically?

Because:

```
axis=0 collapses rows
```

Result:

```
One value per column
```

---

axis=1

```
np.sum(arr_2d, axis=1)
```

Output:

```
[ 6 15 24]
```

Calculation:

```
1+2+3 = 6
4+5+6 = 15
7+8+9 = 24
```

Why axis=1 sums horizontally?

Because:

```
axis=1 collapses columns
```

Result:

```
One value per row
```

3D Arrays and Axis

```
arr_3d = np.array([
    [[1,2,3],[4,5,6]],
    [[7,8,9],[10,11,12]]
])
```

Shape:

```
(2,2,3)
```

Meaning:

```
2 pages, 2 rows, 3 columns
```

---

axis=0 in 3D

```
np.sum(arr_3d, axis=0)
```

Output:

```
[[ 8 10 12]
 [14 16 18]]
```

Meaning:

```
Collapse pages
```

---

axis=1 in 3D

```
np.sum(arr_3d, axis=1)
```

Meaning:

```
Collapse rows
```

---

axis=2 in 3D

```
np.sum(arr_3d, axis=2)
```

Meaning:

```
Collapse columns
```

---

13. Statistical Operations

```
arr = np.array([1,2,3,4,5,6,7,8,9,10])
```

Sum

```
np.sum(arr)
```

Mean

```
np.mean(arr)
```

Average value.

Median

```
np.median(arr)
```

Middle value.

Standard Deviation

```
np.std(arr)
```

Measures spread.

Variance

```
np.var(arr)
```

Spread squared.

Min and Max

```
np.min(arr)
np.max(arr)
```

---

argmin and argmax

```
np.argmin(arr)
np.argmax(arr)
```

Important:

These return:

```
INDEX
```

not actual values.

2D Statistics

```
matrix = np.array([
    [1,2,3],
    [4,5,6],
    [7,8,9]
])
```

```
print(np.sum(matrix, axis=0))
print(np.sum(matrix, axis=1))
```

---

14. Linear Algebra Operations

Very important in:

- Machine learning
- AI
- Graphics
- Physics

Element-wise Multiplication

```
print(a * b)
```

Output:

```
[[ 5 12]
 [21 32]]
```

Dot Product

```
v1 = np.array([1,2,3])
v2 = np.array([4,5,6])

print(np.dot(v1, v2))
```

Output:

```
32
```

Calculation:

```
1×4 + 2×5 + 3×6
```

Why is Dot Product Important?

Used everywhere:

- Neural networks
- Similarity calculations
- Physics
- Machine learning

Determinant

```
matrix = np.array([[1,2],[3,4]])

print(np.linalg.det(matrix))
```

Inverse

```
np.linalg.inv(matrix)
```

Eigenvalues and Eigenvectors

```
np.linalg.eig(matrix)
```

Very important in:

- PCA
- Deep learning
- Computer vision

---

15. Useful Array Methods

Sorting

```
arr = np.array([3,1,4,1,5,9,2,6])

print(np.sort(arr))
```

Output:

```
[1 1 2 3 4 5 6 9]
```

In-place Sort

```
arr.sort()
```

Modifies original array.

Unique Values

```
arr_with_duplicates = np.array([1,2,2,3,3,3,4])

print(np.unique(arr_with_duplicates))
```

Output:

```
[1 2 3 4]
```

Searching with `np.where()`

```
arr = np.array([1,2,3,4,5,6,7,8,9,10])

print(np.where(arr > 5))
```

Output:

```
(array([5, 6, 7, 8, 9]),)
```

Why does `np.where()` return indices?

Because its main purpose is:

```
Locate positions matching condition
```

To get values:

```
arr[np.where(arr > 5)]
```

Output:

```
[ 6  7  8  9 10]
```

Stacking Arrays

```
a = np.array([1,2,3])
b = np.array([4,5,6])
```

Vertical Stack

```
np.vstack((a,b))
```

Output:

```
[[1 2 3]
 [4 5 6]]
```

Horizontal Stack

```
np.hstack((a,b))
```

Output:

```
[1 2 3 4 5 6]
```

Column Stack

```
np.column_stack((a,b))
```

Output:

```
[[1 4]
 [2 5]
 [3 6]]
```

Why Different Stack Methods?

Because data shapes matter.

Different ML/data tasks require:

- Adding rows
- Adding columns
- Merging datasets

---

16. Practical Example: Image Processing Simulation

Images are basically:

```
2D or 3D NumPy arrays
```

Simulating Image

```
image = np.random.randint(0, 256, size=(10,10))
```

Explanation:

Part	Meaning
0	minimum pixel value
256	maximum + 1
size=(10,10)	image dimensions

Image Statistics

```
print(np.mean(image))
print(np.std(image))
print(np.min(image))
print(np.max(image))
```

Used for:

- Brightness
- Contrast
- Pixel analysis

Brightness Increase

```
brighter = np.clip(image + 50, 0, 255)
```

Why `np.clip()`?

Pixel values must stay between:

```
0 to 255
```

Without clipping:

```
Values may exceed valid image range.
```

Contrast Adjustment

```
contrast = np.clip(image * 1.5, 0, 255)
```

Multiplying increases contrast.

---

17. Practical Dataset Analysis

```
scores = np.array([
    [85,90,88],
    [92,87,91],
    [78,82,80],
    [95,98,96],
```

```
    [88,85,90]
])
```

Meaning:

```
5 students, 3 tests
```

Student Averages

```
student_averages = np.mean(scores, axis=1)
print(student_averages)
```

Why `axis=1`?

Because:

```
We want average across columns/tests.
```

Result:

```
One average per student(row)
```

Test Averages

```
test_averages = np.mean(scores, axis=0)
```

Why `axis=0`?

Because:

```
We want average down rows/students.
```

Result:

```
One average per test(column)
```

Best Student

```
np.argmax(student_averages)
```

Returns index of best student.

Excellent Students using Boolean Masking

```
excellent = student_averages > 90
print(excellent)
```

Output:

```
[False False False True False]
```

Why Boolean Masks are Powerful?

They allow:

- Fast filtering
- Conditional selection
- Vectorized data analysis

without loops.

---

18. NumPy Quick Reference Cheat Sheet

Array Creation

```
np.array()
np.arange()
np.linspace()
np.zeros()
np.ones()
np.full()
np.eye()
np.random.rand()
np.random.randint()
```

Array Information

```
.shape
.size
.ndim
.dtype
.itemsize
.nbytes
```

Array Manipulation

```
.reshape()
.flatten()
.T
np.concatenate()
np.vstack()
np.hstack()
```

Mathematical Operations

```
+
-
*
/
**
%
np.sqrt()
```

Statistical Functions

```
np.sum()
np.mean()
np.median()
np.std()
np.var()
np.min()
np.max()
np.argmin()
np.argmax()
```

Linear Algebra

```
np.dot()
np.linalg.det()
np.linalg.inv()
np.linalg.eig()
```

Searching and Sorting

```
np.where()
np.sort()
np.unique()
```

FINAL IMPORTANT INTERVIEW POINTS

Difference Between Python List and NumPy Array

Python List	NumPy Array
Slower	Faster
Mixed types allowed	Same datatype
More memory	Less memory
No vectorization	Vectorized
Loops required	No loops needed

Most Important NumPy Concepts

1. Vectorization
2. Broadcasting
3. Shape
4. Axis
5. Boolean Indexing
6. Reshaping
7. ndarray

Most Common Beginner Mistakes

Mistake 1

Confusing:

```
*
```

with matrix multiplication.

Remember:

```
* → element-wise
np.dot() → matrix multiplication
```

Mistake 2

Confusing axis.

Remember:

```
axis = dimension being collapsed
```

Mistake 3

Wrong reshape size.

Remember:

```
Total elements must stay same.
```

Real-World Applications of NumPy

- Machine Learning
- Deep Learning
- Computer Vision
- Image Processing
- Scientific Computing
- Data Analysis
- Finance
- Robotics
- AI Systems

Conclusion

NumPy is the foundation of:

- Pandas
- Scikit-learn
- TensorFlow
- PyTorch
- OpenCV
- SciPy

If you master NumPy:

you build strong foundations for:

- Data Science
- Machine Learning
- AI
- Scientific Programming

END OF NOTES